

Credenciais e segurança

Regra do repositório

Nenhum segredo real (senha, chave de API, service role key) fica gravado em nenhum arquivo versionado neste repositório — nem em código, nem em documentação, nem em scripts de exemplo. Um segredo commitado num repositório Git continua no histórico pra sempre, mesmo que o arquivo seja apagado depois ou o repositório vire privado no futuro. Por isso este documento explica **onde** cada credencial mora e **como obtê-la/trocá-la**, nunca qual é o valor atual.

Isso é reforçado pelo [.gitignore](#), que já exclui:

```
.env
.env.local
.env.*.local
*credential*
*service_role*
*.pem
*.key
```

Antes de fazer commit de qualquer coisa, confira com `git status` que nenhum desses arquivos aparece como "to be committed".

Inventário de credenciais

Credencial	Onde fica	Quem/o que usa	Nível de sensibilidade
VITE_SUPABASE_URL	.env.local (raiz do projeto, não versionado)	Frontend (browser)	Baixo — é só o endereço do projeto
VITE_SUPABASE_ANON_KEY	.env.local	Frontend (browser)	Baixo/médio — pública por design, mas protegida pela RLS do banco; ainda assim não deixamos hardcoded em código
SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY	.env.migration.local (raiz, não versionado)	Scripts de migração (scripts/migration/load.mjs)	Alto — a service role key ignora toda RLS. Nunca deve ir para o frontend nem para qualquer arquivo versionado
SUPABASE_SERVICE_ROLE_KEY (mesma chave)	Secrets da Edge Function, configurados no painel do Supabase (Project Settings → Edge Functions → Secrets)	Edge Functions admin-users e process-invoice	Alto — injetada automaticamente pelo Supabase no runtime da function; o código nunca contém o valor
GEMINI_API_KEY	Secret da Edge Function process-invoice, configurado no painel do Supabase	Edge Function process-invoice (leitura de faturas por IA)	Alto
Login do usuário MASTER inicial	Criado diretamente no Supabase Auth (Authentication → Users) do projeto em uso	Login na aplicação	Alto — é a conta com acesso total

Como configurar o frontend (.env.local)

```
cp .env.example .env.local
```

Preencha com os valores do **seu** projeto Supabase, em Project Settings → API:

- `VITE_SUPABASE_URL` → campo "Project URL"
- `VITE_SUPABASE_ANON_KEY` → campo "anon public" (ou a chave `sb_publishable_...`, no formato novo de chaves do Supabase) — **nunca** a `service_role`.

Como configurar os scripts de migração (`.env.migration.local`)

Só necessário se for rodar o ETL de `scripts/migration/` de novo (migração pontual, não faz parte do uso normal do app):

```
SUPABASE_URL=https://<seu-projeto>.supabase.co
SUPABASE_SERVICE_ROLE_KEY=<pegue em Project Settings → API → service_role>
```

A `service_role` **ignora toda Row Level Security** — trate como uma senha de root do banco. Nunca cole esse valor num chat, num commit, ou em qualquer lugar fora desse arquivo local.

Usuário MASTER (acesso total à aplicação)

O papel `MASTER` é o único que pode aprovar/rejeitar/bloquear outros usuários, trocar papéis, ver a auditoria completa e administrar contas. Ele é criado de duas formas:

1. **Bootstrap automático:** a função `handle_new_user()` (banco de dados — ver [banco-de-dados.md](#)) promove automaticamente a `MASTER / APPROVED` o primeiro usuário cujo e-mail bata com o e-mail configurado dentro da própria função SQL. Esse e-mail fica no código-fonte da migration (é um identificador, não um segredo), mas a **senha** desse usuário nunca é definida pelo código — é configurada manualmente por quem cria a conta.
2. **Promoção manual:** qualquer MASTER existente pode promover outro usuário pela RPC `change_user_role`.

Pra criar/trocar a senha do usuário MASTER do projeto que você está operando:

1. Supabase Dashboard → Authentication → Users.
2. Encontre o usuário pelo e-mail configurado como bootstrap (veja a função `handle_new_user` na migration `20260811140000_app_profiles.sql`).
3. Use o menu do usuário para "Send password recovery" (o próprio usuário troca) ou, como MASTER logado no app, use a tela de administração de usuários (que chama a Edge Function `admin-users`, ação `reset_password`).

Este repositório não guarda a senha de nenhuma conta real. Se você precisa compartilhar essa senha com alguém do time, use um gerenciador de senhas (1Password, Bitwarden etc.) — nunca chat, e-mail ou um arquivo de texto dentro do projeto.

Rotacionando uma credencial vazada

Se vazou...	Faça isso
VITE_SUPABASE_ANON_KEY	Baixo risco por si só (é protegida por RLS), mas se quiser trocar: Project Settings → API → gerar nova chave anon; atualize <code>.env.local</code>
SUPABASE_SERVICE_ROLE_KEY	Urgente: Project Settings → API → gerar nova service role key; atualize <code>.env.migration.local</code> e os secrets das Edge Functions; a chave antiga para de funcionar imediatamente
GEMINI_API_KEY	Revogue no Google AI Studio / Google Cloud Console e gere outra; atualize o secret da Edge Function <code>process-invoice</code>
Senha de um usuário (inclusive MASTER)	Reset pela tela de administração (MASTER) ou pelo fluxo "Esqueci minha senha" do próprio login

Edge Functions e por que existem

`admin-users` e `process-invoice` (ver [arquitetura.md](#)) existem porque algumas operações **exigem** a service role key (criar/excluir usuário em `auth.users`, ler um PDF do Storage com um bucket restrito) e essa chave nunca pode chegar ao browser. As functions rodam no servidor do Supabase, recebem a chave via variável de ambiente injetada automaticamente (não hardcoded), conferem que quem chamou é um MASTER aprovado e só então usam o cliente privilegiado.